LAYE FOODS

Website + Food Ordering System • Full-Stack AI Build Prompt (Maximum 2 Pages)

ROLE & OBJECTIVE

Act as a senior full-stack engineer, UX/UI designer, database architect, QA tester, and AI integration specialist. Build **Laye Foods**, a polished, production-minded restaurant website and online food-ordering platform for customers in Nigeria. Do not create only a visual mock-up: implement working frontend, backend, database operations, order workflows, and an integrated AI chatbot. Make sensible assumptions where details are missing, document them, and keep secrets out of browser code.

1. FRONTEND, UI & PAGES

- **Design:** modern, appetising, mobile-first interface; warm food photography; a clean palette such as deep green, cream, and a tasteful orange accent; readable typography, consistent spacing, accessible contrast, clear focus states, and subtle motion. Use responsive layouts for phones, tablets, and desktop.
- **Navigation:** logo, Home, Menu, About, Contact, Cart, account/login, and a prominent “Order Now” call-to-action. Show cart item count and clear loading, empty, success, and error states.
- **Required pages:** Home (hero, popular meals, offers, how ordering works, testimonials), Menu/Products (search, categories, filters, availability, meal details and add-ons), Cart, Checkout, Order Confirmation/Tracking, About, Contact/FAQ, Login/Register, Customer Dashboard, and protected Admin Dashboard.
- **Menu & cart:** show food name, description, image, price in NGN, category, availability, portion/add-ons, and dietary/allergen notes when known. Let users add/remove items, change quantities, see subtotal, delivery fee, discount and total, and keep cart state reliable across navigation.
- **Accessibility & quality:** semantic HTML, keyboard navigation, form labels, alt text, responsive images, validation messages, useful page titles, and no dead links or placeholder buttons. Provide realistic sample menu data only if clearly marked as seed/demo data.

2. BACKEND, SUPABASE & DATA MODEL

Use Supabase for PostgreSQL database, authentication, and image storage where appropriate. Connect the real UI to the database; never fake successful saves or hard-code live order data. Create documented schema/migrations and relationships for: **profiles/users, categories, menu_items, item_options/add-ons, carts (optional), orders, order_items, addresses, payments/payment_attempts, delivery details, contact_messages, chatbot_conversations/messages, and settings/promotions.** Include IDs, timestamps, status fields, price snapshots on order items, sensible indexes, and constraints.

- **CRUD:** implement create/read/update/delete where appropriate for menu items, categories, user profiles/addresses, messages, and admin-managed content. For orders, prefer status transitions and audit history over hard deletion.
- **Security:** enable Supabase Row Level Security (RLS). Customers can read/update only their own permitted profile data and view only their own orders; admins can manage menu and orders through verified roles and server-side authorization. Never trust a client-supplied price, role, discount, or order status.
- **Integration:** configure environment variables for Supabase URL and public anon key; keep service-role keys, payment secrets, and AI keys server-side only. Include `.env.example`, setup instructions, seed data, and clear errors for missing configuration.

3. ORDER LOGIC & PROBLEM-SOLVING

- Build a robust checkout flow: validate customer/contact details, delivery or pickup choice, delivery address/area, opening hours, item availability, quantities, and current prices on the server before creating an order.
- Calculate totals on the server from trusted menu records: item subtotal + approved add-ons + delivery fee – valid discount. Store a snapshot of item names, options, quantities, and prices so later menu edits do not change old orders.
- Use a database transaction or secure server-side operation to create the order and its order items consistently. Prevent duplicate orders from double-clicks/retries with an idempotency key; disable repeat submission while processing and show a recoverable error if the request fails.
- Use explicit statuses such as pending, confirmed, preparing, ready_for_pickup/out_for_delivery, delivered, and cancelled. Validate allowed transitions, record timestamps, and show customers only appropriate updates. Handle out-of-stock items, closed hours, invalid addresses, failed payments, cancellations, and admin corrections gracefully.
- Support payment-on-delivery and/or pickup as an initial option. If online payment is added, use a reputable provider such as Paystack or Flutterwave, verify payment status on the server/webhook, and never store raw card details. Do not mark an order paid from a browser redirect alone.
- Admin tools: secure role-gated order queue, search/filter by status/date, view order details, update status, manage availability/prices, and review messages. Add audit logs for important admin actions.

4. INTEGRATED AI CHATBOT

Add a friendly Laye Foods assistant accessible from every page. It should answer menu, ingredients/allergens (only from verified data), opening hours, delivery areas/fees, payment methods, and order-status questions. Use a server-side AI API route with secrets stored in environment variables, conversation history handled safely, rate limits, input validation, and a helpful fallback if the AI service is unavailable. Ground answers in current menu/business data; do not invent prices, stock, delivery times, or policies. For personal order lookup, require sign-in or a safe order-verification step and authorize every database request. The bot may help assemble a cart, but must ask the customer to review and explicitly confirm before placing an order; it must never claim an order/payment succeeded without backend confirmation. Provide a clear way to contact a human.

5. DELIVERY REQUIREMENTS & ACCEPTANCE TESTS

- Use a maintainable component structure, typed/validated inputs where supported, reusable UI components, secure server endpoints, and documented setup/deployment steps. Suggested stack: Next.js + TypeScript + Tailwind CSS, Supabase, and a server-side AI provider; choose compatible stable versions and explain key decisions.
- Test on mobile and desktop. Verify registration/login, RLS boundaries, menu CRUD, cart math, server-side price validation, successful and failed checkout, duplicate-submit prevention, order tracking, admin permissions, chatbot fallback, and unavailable/out-of-stock items.
- Deliver a working end-to-end application, database schema/migrations, `.env.example`, seed/demo data, setup instructions, and a concise list of integrations and remaining configuration. Do not expose secrets, skip authorization, leave core features as TODOs, or report tests as passed unless they were actually run.

Final instruction: Build in stages—inspect requirements, design the schema and order states, implement the UI and backend, connect Supabase, integrate the chatbot, then run tests and fix failures. Prioritise correctness, security, clear customer feedback, and a smooth ordering experience over decorative effects.